Comparative Evaluation of File-Upload Recovery Strategies under Controlled Network Disruption

Sreerag Aneesh Nair (24BCE0081) and Rini Vaishnavi R S (24BAI0261)
Vellore Institute of Technology
BCSE308P Computer Networks Lab | Team 24 | Theory F1+TF1 | Lab L35+L36 | October 2026

***Abstract*—**Unstable connectivity can cause a nearly complete file upload to be discarded. This study implements a local experimental platform that compares Basic, Retry and Resumable upload strategies using HTTP body transfers over TCP sockets. A controllable proxy applies a shared bandwidth limit, a configurable delay before each request, and scheduled connection outages. Each trial receives a fresh upload origin and an independent scenario clock. The evaluation uses one deterministic payload of 524,288 bytes, three scenarios and three trials per strategy, giving 27 uploads. All 18 uploads conducted under stable and slow conditions completed with matching SHA-256 digests. During a five-second outage, the Basic strategy failed in all three trials, whereas the Retry and Resumable strategies each completed all three. Retry required an average of 9.95 seconds and transferred 498,347 redundant application bytes. Resumable required an average of 8.50 seconds and transferred 4,096 redundant application bytes, a reduction of 99.2% for the tested settings. The principal contribution is a reproducible implementation and a fair experimental comparison of established recovery methods. The results also show the request overhead introduced by chunking when no outage occurs. The platform measures application-level behaviour through a localhost proxy and does not emulate packet-level conditions or represent real wireless reliability.

Index Terms*—file upload, resumable transfer, network disruption, HTTP, retry strategies, controlled experiment, localhost proxy.*

I. INTRODUCTION

A. Problem statement

A file transfer may encounter limited bandwidth or a temporary loss of connectivity. A client that restarts the entire upload may eventually succeed, but it retransmits data that has already been sent. A resumable client can retain progress that the server has confirmed, at the cost of additional requests and protocol state. The relevant engineering question is which strategy completes the transfer correctly within the available time and with an acceptable volume of redundant application data.

This question is significant for applications that upload assignments, video, backups or field reports over unreliable connections. The laboratory isolates recovery behaviour so that no strategy receives a more favourable network schedule than another. The platform is intended as a development and evaluation tool and is not a public file-storage service.

B. Objectives

- Transfer actual file bytes through a controlled proxy and compare three recovery strategies.
- Apply equivalent schedules independently to each trial, including bandwidth changes and connection interruptions.
- Record completion, retries, recovery time, redundant application bytes and SHA-256 integrity, without derived quality scores.
- Provide configuration controls, live traces, event inspection and data export, together with an automated failure-boundary sweep.

C. Contribution and novelty boundary

Retry and resumable uploads are established techniques. The tus protocol is a standardized example of resumable upload [4]. This study implements a simpler custom HTTP protocol and does not claim conformance with tus or a new transport algorithm. Its contribution is the integration of a shared-rate disruption proxy, server-confirmed recovery, repeatable serial trials and transparent application metrics within a single inspectable laboratory. Section II summarizes the prior work that defines this boundary.

The experimental question is empirical: how much completion time and redundant application data differ among strategies under identical conditions. The exported measurements, verified file hashes, reproducible configuration and stated limitations support the comparison.

II. RELATED WORK

Prior to designing the experiment, the authors reviewed published and preprint work on upload recovery, retry behaviour, configuration-sensitive testing and automated search for network failures. The review was conducted on 24–25 October 2026. Its purpose was to avoid claims of novelty that existing work already addresses and to position the laboratory accurately. The review is a literature audit only; it does not repeat any experiment.

A. Summary of findings

The review did not support a broad novelty claim. Searching for network conditions in which one recovery policy fails while another succeeds, comparing against a calculated reference, and using failure cases to guide repairs are all addressed by existing work [5]–[7]. Comparing configurations through differential performance feedback is also established [8]. Recovery methods based on fragmentation and checkpointing have analytical treatments [9]. Retry defects and configuration-aware fault testing have been studied [10]–[12].

Accordingly, the contribution of this study is an integrated, inspectable testbed for comparing established recovery methods under identical, replayable disruption schedules. It does not propose a new recovery algorithm.

B. Comparison with prior work

TABLE I
PRIOR WORK REVIEWED AND ITS RELEVANCE TO THIS STUDY

Source	Principal findings	Relevance to this study
CC-Fuzz, HotNets 2022 [5]	Proposes differential trace search, in which one congestion controller performs poorly while others perform well, with infrequent reference evaluations to reduce cost.	Comparative search for avoidable network failures is already established and is not claimed as a contribution.
AdvNet, CoNEXT2 2026 [6]	Optimizes the performance gap between a calculated reference or executable protocol and a tested protocol, using sample-efficient search that tolerates noisy emulation.	The combination of an ideal reference, an executable comparator and noise-aware search is not an unexplored area.
ReGuard, May 2026 preprint [7]	Applies bilevel regret search across shared scenarios and derives corrective rules from counterfactual trajectories, with conditional theoretical guarantees.	The discover-explain-repair loop is covered by this work. As a preprint, it is not treated as peer-reviewed.
Differential performance fuzzing of configuration options, SBFT 2025 [8]	Uses differential performance feedback to generate inputs that reveal unexpected performance relationships between configurations.	Configuration comparison with differential feedback is established. This summary relies on the publisher abstract and introduction.
Channel failures and file fragmentation policies [9]	Models fragments with overhead and interruptions, analyzes completion time and fragmentation policies, and extends the analysis to checkpointing.	Adaptive chunk sizing and the loss of incomplete work have analytical precedents.
Cast, ICSE-SEIP 2026 [10]	Applies performance thresholds to test judgments and reports erroneous judgments, including acceptable degradation.	Motivates careful pass/fail criteria in performance testing.
Wasabi, SOSP 2024 [11]	Studies retry timing and caps, and observes that configuration-dependent retry defects may go undetected without the problematic setting.	Supports testing under specific configurations. It does not establish that configuration-aware testing is new.
CAFault, USENIX ATC 2025 [12]	Combines configuration inputs with fault inputs using a fault-dependent model and fault-handling feedback.	Configuration-aware fault testing is already addressed in prior work.
tus resumable upload protocol [4]	Standardizes offset-based resumable uploads.	Prior art for offset-based recovery. This laboratory uses a simpler custom protocol.

C. Persistence contracts in practice

Commercial services implement different persistence rules, so the atomic commit model of this laboratory represents one contract rather than a general resumable-upload behaviour. Amazon S3 multipart uploads require every part except the last to be at least 5 MiB [13], and interrupted parts are retried individually [14]. Google

Cloud resumable uploads can resume from data the service has already persisted within an interrupted transfer [15].

The laboratory discards incomplete request bodies, which corresponds to neither contract exactly. Any adapter to a real service would need to reproduce that service's persistence rules, including its chunk-size requirements.

D. Claim status

- Not claimed as novel: automatic search for avoidable network failures; fault-driven configuration comparison; use of a calculated reference; the discover-and-repair cycle; and a three-way classification of upload outcomes considered in isolation.
- Claimed: an integrated laboratory combining a shared-rate disruption proxy, server-confirmed recovery, replayable serial trials and transparent application metrics, applied to a fair comparison of Basic, Retry and Resumable uploads.
- Not yet established: the superiority of any deployable upload policy. Such a claim requires held-out experiments and independent implementations.

E. Open research direction

The review identified a possible follow-on question: whether a protocol-aware feasibility bound can identify infeasible upload scenarios more effectively than raw-capacity filtering. Such a bound would estimate the maximum data that can be committed before a deadline under the upload's atomic and chunking rules, allowing scenarios that cannot succeed to be excluded before costly live trials.

The novelty of this direction has not yet been verified. A per-trace dynamic program may correspond to a known interrupted-task or checkpoint-scheduling method, and the prior work must be examined before any novelty claim is made. This direction is listed under future work in Section XII.

III. SYSTEM ARCHITECTURE

A. Measured data path

TABLE II
PRINCIPAL COMPONENTS OF THE EXPERIMENTAL PLATFORM

Component	Responsibility
Python upload client	Reads the payload and executes Basic, Retry or Resumable HTTP requests.
Disruption proxy	Forwards body bytes under shared pacing, delays each request and aborts connections during outages.
Upload origin	Commits complete requests atomically, reports contiguous committed offsets and computes the digest of the completed file.
Dashboard and scheduler	Serve the local interface, execute isolated trials serially, publish live state and export results.

The browser sends control requests to the laboratory API. The Python client opens a TCP connection to the proxy, which opens a separate TCP connection to the origin. All services bind to 127.0.0.1. Proxy and origin ports are assigned ephemerally for each trial, and the dashboard listens on port 8765 by default. The networking primitives are provided by Python asyncio streams [1].

The initial file selection in the browser is performed before the measured experiment begins. The Python client then reuses the same bytes for each strategy. Generated payloads repeat byte values from 0 through 255. Files and partial origin bodies are held in memory, subject to a 32 MiB payload limit.

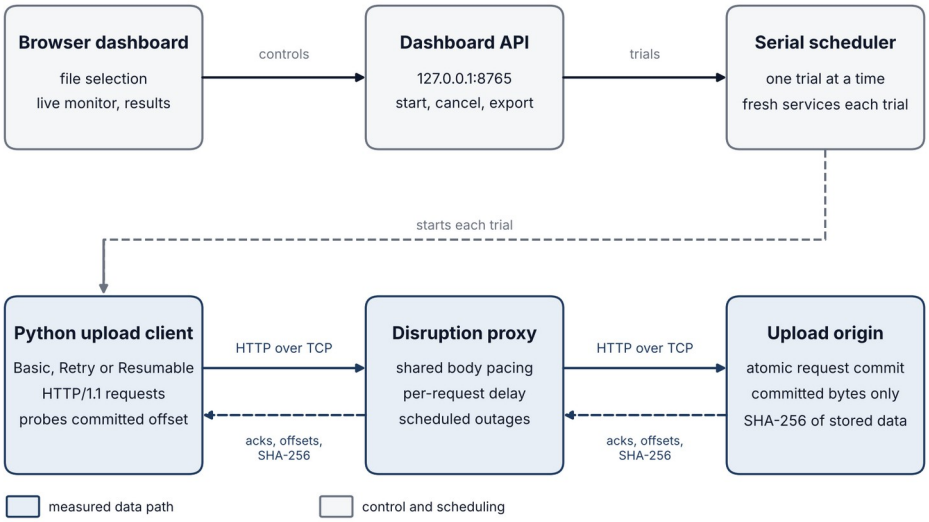


Fig. 1. Architecture of the experiment. The control path (grey) starts each serial trial with fresh origin and proxy services. The measured path (navy) carries HTTP body bytes from the upload client through the disruption proxy to the upload origin, and acknowledgements and offsets return along the same route. The browser file selection occurs outside experiment timing and bypasses the proxy.

B. Implementation map

TABLE III
SOURCE MODULES AND THEIR RESPONSIBILITIES

Source	Purpose
main.py	Command-line launcher and service shutdown.
app.py	Static and control server, serial scheduling, failure-boundary sweeps and CSV/JSON export.
throttler.py	Upload origin, proxy pacing, phase monitor, recovery clients and cleanup.
config.py / metrics.py	Scenario bounds and exact metric definitions.
static/ and tests/	Interface controls and charts; integration tests and demonstration runner.

IV. UPLOAD STRATEGIES AND PROTOCOL BEHAVIOR

A. Basic

The Basic strategy transmits the entire payload in a single HTTP request. The origin commits the file only after receiving the complete body. A connection failure or server error terminates the trial without retry. Confirmed progress may remain at zero while the proxy forwards bytes, because the full request is committed atomically.

B. Retry

The Retry strategy transmits complete-file requests from offset zero. Upon failure, it waits with exponential backoff and restarts the payload. Backoff begins at 0.25 seconds and then increases to 0.5, 1 and 2 seconds, with a maximum of 2 seconds. The retry budget applies to the entire trial. A new connection does not reset the scenario clock or the pacing allowance.

C. Resumable

The Resumable strategy transmits sequential chunks, 32 KiB by default. An acknowledged complete chunk becomes committed progress. After a failure, the client waits, queries the origin for its committed offset through the same proxy, and continues from that offset. An incomplete chunk is retransmitted, because the origin discards incomplete request bodies.

TABLE IV
BEHAVIORAL PROPERTIES OF THE THREE UPLOAD STRATEGIES

Property	Basic	Retry	Resumable
Request unit	Whole file	Whole file	One chunk
Response to failure	Terminate trial	Restart from zero	Query committed offset
Commit boundary	Whole file	Whole file	Complete chunk
Additional work	Minimal	Backoff and restarts	Chunks, offset queries and backoff

D. Integrity verification

The final successful response contains a SHA-256 digest computed over the committed bytes at the origin. The client compares this digest with a digest of the original payload, computed using hashlib [2]. A mismatch is recorded as a failure. An interrupted or unverified file has no integrity result and is never recorded as a success.

The custom protocol uses HTTP/1.1. POST requests carry data together with a byte offset, and GET requests retrieve the origin's committed progress. HTTP provides the framing and status semantics [3]. This version preserves progress across connections within a single trial, but not across restarts of the origin process.

V. PROXY CONTROLS AND USER INTERFACE

A. Controlled network behaviour

Bandwidth is expressed in decimal kilobits per second, such that bytes per second equal kbps × 1,000 / 8. A shared scheduler paces the upload body in slices of up to 4 KiB. All connections within a trial share this scheduler, so that reconnection does not create independent bandwidth allowances. Oversleep credit in the scheduler is shared and limited to one slice.

The delay is a single wait applied before each HTTP request is forwarded, including offset queries. Headers and small responses are excluded from the bandwidth limit. The delay does not represent a guaranteed round-trip time. During an outage, the proxy aborts active connections and rejects new requests. It does not inject packet loss and does not modify host network settings.

B. Interface controls

TABLE V
INTERFACE CONTROL GROUPS AND THEIR FUNCTIONS

Control group	Function
Payload	Selects a local file, or generates deterministic bytes of a specified size.
Scenario	Selects stable, low-bandwidth, sudden-slowdown, temporary-disconnect, intermittent-outage, demonstration or custom phase schedules.
Strategies and trials	Selects the strategies and the number of repetitions. Starting an experiment runs the strategies separately; cancellation stops the current and all queued work.
Advanced settings	Specifies the request delay, the total retry budget and the resumable chunk size.
Live monitor	Displays elapsed time, forwarding throughput, the applied limit, confirmed bytes and events.
Results	Displays the outcome, duration, useful throughput, extra bytes, retries, recovery, integrity, exports and row-level inspection.

C. Failure-boundary mode

The sweep varies bandwidth and outage duration for a specified file size, deadline and number of trials. Each cell begins its outage at one second and restores the configured rate afterward. The sweep reports verified success rates, excluding cancelled trials. A small sweep was functionally verified during Step 1. The 27-trial evaluation reported here uses three named scenarios rather than a full sweep. Cells that complete before an outage occurs do not test recovery.

VI. EXPERIMENTAL DESIGN

The design comprises three strategies, three trials and three scenarios, yielding 27 uploads. Within each trial number, the strategies are executed in the order Basic, Retry, Resumable. The scenarios were collected in the order stable, slow, disconnected. The order is fixed rather than randomized.

TABLE VI
PARAMETERS HELD CONSTANT ACROSS ALL EVALUATION GROUPS

Parameter	Value
Payload	524,288 bytes (0.5 MiB), deterministic byte sequence
Strategies and repetitions	Basic, Retry and Resumable; three trials each
Deadline	60 seconds per trial
Request delay	20 ms before each forwarded request
Maximum retries	12 additional attempts per trial
Resumable chunk size	32,768 bytes (32 KiB)
Environment	Windows, Python 3.13.13, localhost HTTP over TCP
Observation source	Dashboard exports and browser captures

TABLE VII
BANDWIDTH SCHEDULES OF THE THREE SCENARIOS

Scenario	Bandwidth schedule from the start of each trial
Stable	2,048 kbps throughout.
Slow	256 kbps throughout.
Disconnected	2,048 kbps initially; offline from t = 2 s to t = 7 s; restored to 2,048 kbps.

The nominal forwarding time is 524,288 × 8 / 2,048,000 = 2.048 seconds. The two-second interruption therefore begins before the payload has been fully transmitted. Event logs confirm that every disconnected trial encountered the outage.

A. Fairness and repeatability

Each trial creates fresh proxy and origin services and resets the clock, counters, committed bytes and retry budget. Strategies do not compete for bandwidth, and all groups use the same payload hash. Exports preserve the settings, samples and events. Three repetitions describe variation but do not establish long-term reliability. Machine load, the fixed order of execution and the local software stack remain possible confounding factors.

VII. METRIC DEFINITIONS

The application records observed outcomes rather than subjective ratings. Values that cannot be measured are reported as unavailable. The definitions below apply to both the interface and the exported results.

TABLE VIII
METRIC DEFINITIONS AND POINTS OF OBSERVATION

Metric	Definition and point of observation
Outcome and integrity	Completion requires matching SHA-256 digests for the original payload and the committed origin bytes. Other terminal states are failed or cancelled.
Elapsed time	Monotonic time from the start of the trial to hash verification or terminal error. Includes delays, retries, offset queries and backoff.
Proxy ingress bytes	Body bytes read by the proxy, including redundant transmissions. Excludes headers and bytes remaining in unread client buffers.
Client-submitted bytes	Body bytes passed to the client socket. May include buffered bytes that never reached the proxy.
Forwarded bytes	Body bytes submitted by the proxy to the origin socket. This is not a wire-level delivery count.
Confirmed bytes	Contiguous committed bytes at the origin. Acknowledgements supply live values; final cleanup reads the origin state.

Metric	Definition and point of observation
Redundant application bytes	max(0, proxy ingress bytes − final committed unique bytes). Incomplete atomic uploads are counted as discarded work.
Overhead percentage	100 × redundant application bytes / payload bytes, for verified successes only.
Useful throughput	Verified payload bytes × 8 / elapsed seconds / 1,000, in kbps. Unavailable for incomplete files.
Retry count	Additional attempts begun after a failure and backoff. Successful chunks and offset queries are not counted as retries.
Recovery time	Interval from scheduled restoration of connectivity to the next acknowledgement that increases confirmed progress. For Retry this is normally full-file completion; for Resumable it may be a chunk acknowledgement.
Success rate	Verified completions divided by completed, non-cancelled trials within the same configuration group.
Transport overhead	Unavailable. TCP/IP headers, wire-level bytes, packet loss and TCP retransmissions are not measured.

VIII. RESULTS ACROSS ALL SCENARIOS

The results table reports three trials per row. Means and sample standard deviations of completion time are calculated from verified successes only. Redundant bytes are averaged across all trials, including failed Basic uploads. N/A denotes an unavailable completion time or recovery value.

TABLE IX
OBSERVED RESULTS FOR ALL SCENARIOS (THREE TRIALS PER ROW)

Scenario	Strategy	Verified	Mean ± SD (s)	Mean redundant bytes
Stable	Basic	3/3	2.12 ± 0.010	0
Stable	Retry	3/3	2.12 ± 0.003	0
Stable	Resumable	3/3	2.61 ± 0.006	0
Slow	Basic	3/3	16.46 ± 0.009	0
Slow	Retry	3/3	16.46 ± 0.002	0
Slow	Resumable	3/3	16.95 ± 0.005	0
Outage	Basic	0/3	N/A	496,981
Outage	Retry	3/3	9.95 ± 0.006	498,347
Outage	Resumable	3/3	8.50 ± 0.007	4,096

Sample standard deviations describe these three trials and are not confidence intervals. Full-precision values are retained in summary.csv.

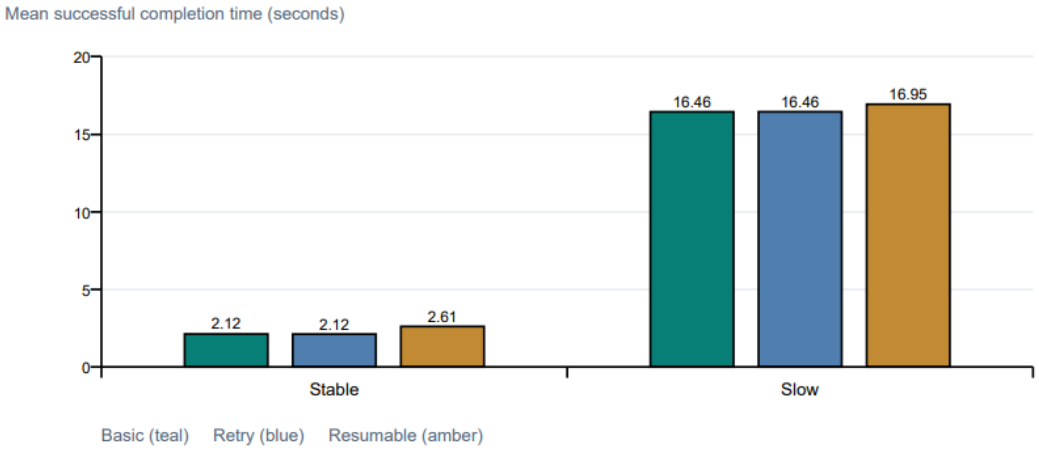


Fig. 2. Mean successful completion time by scenario. Each bar averages three verified uploads. Under stable conditions, Basic required 2.12 seconds; at 256 kbps, it required 16.46 seconds, an increase of 7.75 times against a nominal bandwidth ratio of 8. Resumable required 2.61 and 16.95 seconds respectively, reflecting additional request overhead when recovery is not needed.

IX. RECOVERY AND REDUNDANT-TRANSFER ANALYSIS

The Basic trials in the disconnected scenario failed near the two-second interruption. These failure times are not completion times. Retry and Resumable each completed three of three trials with matching digests.

TABLE X
OUTAGE-SCENARIO METRICS FOR THE TWO SUCCESSFUL STRATEGIES

Outage metric	Retry	Resumable
Mean completion time (successful trials)	9.95 s	8.50 s
Mean observed recovery time	2.95 s	1.01 s
Mean redundant application bytes	498,347 bytes	4,096 bytes
Mean overhead (successful trials)	95.05%	0.78%
Mean additional attempts	5	5

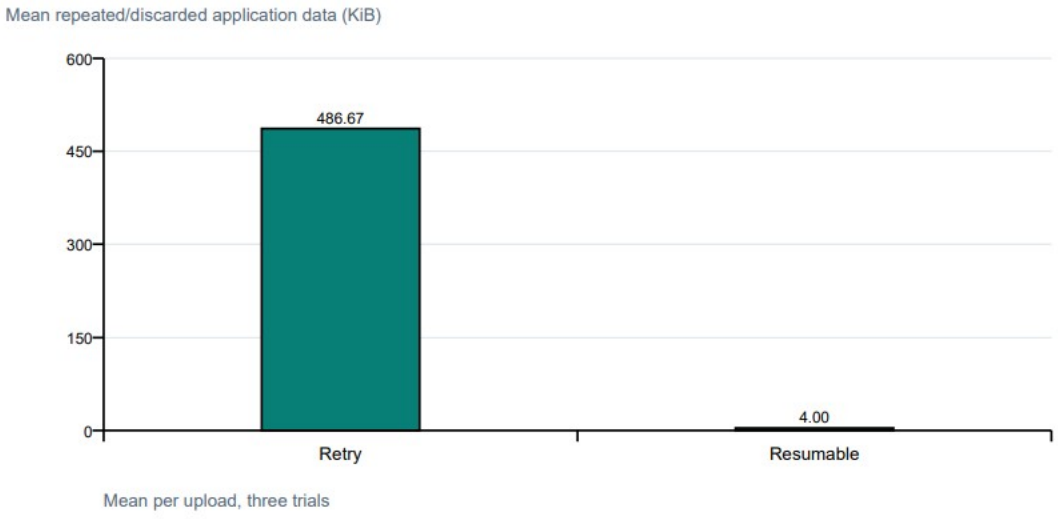


Fig. 3. Both strategies succeed, but restarting repeats substantially more body data. Values are based on proxy ingress rather than TCP retransmissions.

A. Interpretation

Resumable produced 99.2% fewer redundant application bytes than Retry in this experiment. It preserves acknowledged chunks and retransmits only unfinished work, whereas Retry transmits the full file after restoration. Backoff accounts for the fact that useful progress does not resume at exactly seven seconds.

The two strategies acknowledge progress at different granularities: Retry at full-file completion, and Resumable at each committed chunk. This distinction must accompany any comparison of recovery times. The measurements support a workload-specific advantage under the chosen outage pattern. They do not establish that resumable uploads are universally faster.

X. DASHBOARD OBSERVATIONS

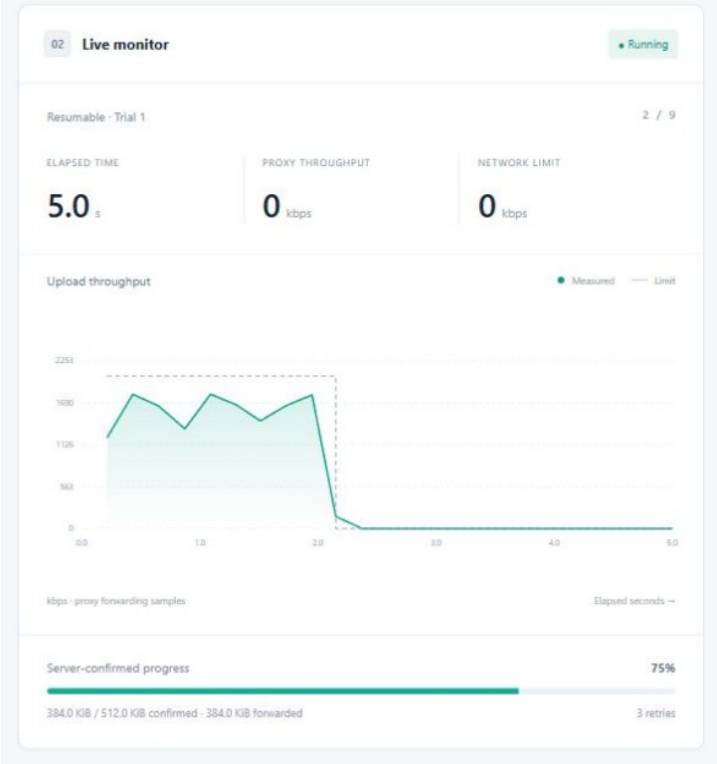


Fig. 4. Resumable trial 1 during the outage, as recorded by the live monitor. Measured throughput and the applied limit fall to zero while 384 KiB of confirmed progress remains. At this point, server-confirmed progress was 75%, with three retries recorded. The image was cropped from the original screenshot without altering any values.

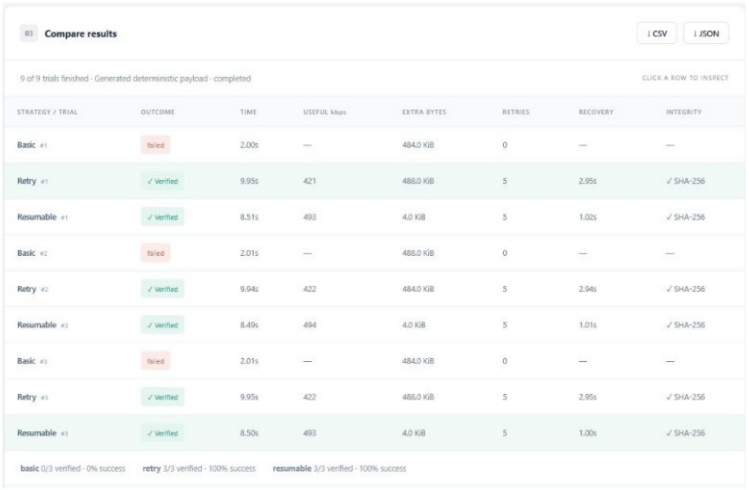


Fig. 5. Result table for the outage scenario. The Basic failures are the expected outcome for that strategy. Retry and Resumable verified all trials. Full-resolution screenshots accompany the repository.

XI. VALIDATION AND ROBUSTNESS

The final Step 1 integration run passed 18 tests in 26.106 seconds. The tests exercise real localhost sockets and wall-clock pacing. JavaScript syntax checks and Python compilation also completed successfully. These statements describe checks that were actually executed; no remote-network test is claimed.

TABLE XI
EXECUTED VALIDATION COVERAGE

Test area	Coverage
Transfers and throttling	Real body bytes, matching digests, bandwidth-dependent durations and shared limits across concurrent connections.
Recovery	Basic interruption failure, Retry restart overhead, Resumable operation from nonzero offsets, repeated outages and changing bandwidth.
Bounds and errors	Retry exhaustion, origin errors, corrupted digests, request delay, deadlines and invalid settings.
Scheduler and interface	Serial repetitions, identical payloads, file selection, exports, cross-origin rejection and cancellation before scheduling.
Cleanup	Large buffered uploads and ordinary cancellation close sockets, clear partial data and stop queued work.

A. Defect identified during verification

Browser testing revealed a cancellation defect that was not detected by the original tests, which used smaller payloads. A socket-close timeout replaced a cancellation exception that was already propagating, so the scheduler recorded a failure and continued to the next strategy. A regression test reproduced the defect before correction. Separating cancellation from ordinary cleanup errors preserved the original exception, and the regression test and the full suite passed afterward.

B. Interface and sweep checks

Browser checks covered theme persistence, validation of empty strategy selections, file selection, row inspection, custom phase schedules, advanced settings, downloads and cancellation. Parsed CSV and JSON exports confirmed the counts and outcomes. A separate six-trial boundary sweep in Step 1 produced successful uploads without an outage and the expected Basic failure in the outage cell. This is a functional check rather than a comprehensive boundary study.

The Step 2 analysis validated all 27 records, identical payload digests, consistent settings, complete trial numbering, matching CSV and JSON rows, and equality of digests for successfully transferred files. A checksum manifest covers the supporting evidence files.

XII. LIMITATIONS, CONCLUSION AND REPRODUCTION

A. Limitations

- All measured transfers were conducted on a single computer through a user-space proxy. Two-computer, Wi-Fi and mobile-hotspot evaluations have not been performed.
- The proxy controls pacing, request delay and connection interruptions. Genuine packet loss, TCP overhead and complete remote-network behaviour are outside the scope of this study.
- Origin progress is held in memory. Recovery across connections operates within a trial. Durable storage and recovery after a server failure are not implemented.
- Three trials per group, a fixed order of execution, a single payload size and a single chunk size limit the statistical confidence and generalizability of the results.
- The atomic discard of incomplete chunks represents one persistence contract. It does not reproduce the rules of any particular commercial upload service (Section II).

B. Future work

A useful extension is a two-computer demonstration with a separately reachable origin and controlled access. Further evaluation could vary payload and chunk sizes, randomize the order of execution and compare additional traces. Persistent storage would enable a separate server-restart experiment. A protocol-aware feasibility bound, described in Section II, is a candidate follow-on study, subject to verification of its novelty against prior work. These are proposed extensions and are not existing capabilities.

C. Conclusion

This study compares upload recovery using actual, inspectable transfers. Under the measured outage, Basic fails, Retry retransmits the complete file, and Resumable succeeds with substantially fewer redundant application bytes. The stable-condition results also show the cost of chunk acknowledgements. The choice of strategy therefore depends on the interruption pattern and the protocol overhead. The review in Section II indicates that the contribution lies in the integrated and inspectable comparison, rather than in a new recovery method.

D. Reproduction procedure

```
python main.py          # Open http://127.0.0.1:8765
python -m unittest discover -v
python docs/step2-experiments/analyze_results.py
```

The application requires Python 3.11 or later and a current browser; it has no third-party package dependencies. To repeat the evaluation, use a 0.5 MiB payload, all three strategies, three trials, a 60-second deadline, a 20 ms delay, 12 retries and 32 KiB chunks. Run the stable, low-bandwidth and temporary-disconnect scenarios separately, and export each completed experiment.

REFERENCES

[1] Python Software Foundation, “Python 3.13 documentation: asyncio streams.” https://docs.python.org/3.13/library/asyncio-stream.html

[2] Python Software Foundation, “Python 3.13 documentation: hashlib.” https://docs.python.org/3.13/library/hashlib.html

[3] R. Fielding, M. Nottingham and J. Reschke, “HTTP Semantics,” RFC 9110, June 2022. https://www.rfc-editor.org/rfc/rfc9110.html

[4] tus community, “Resumable Upload Protocol,” version 1.0.0. https://tus.io/protocols/resumable-upload

[5] “CC-Fuzz,” HotNets 2022. https://www.cs.cmu.edu/~srini/papers/papers/2022-Ray-hotnets/2022-Ray-hotnets.pdf

[6] “AdvNet,” CoNEXT2 2026 (PACMNET). https://arxiv.org/html/2605.00755v1. DOI: https://doi.org/10.1145/3808660

[7] “ReGuard,” preprint, May 2026 (not peer-reviewed). https://arxiv.org/html/2605.04373v1

[8] “Differential performance fuzzing of configuration options,” SBFT 2025. https://doi.org/10.1109/SBFT66712.2025.00014

[9] “On channel failures, file fragmentation policies, and heavy-tailed completion times.” https://www.ee.iitb.ac.in/~jayakrishnan.nair/papers/Fragmentation_ToN.pdf

[10] “Cast,” ICSE-SEIP 2026. https://arxiv.org/html/2602.00972v1

[11] “Wasabi: Detecting retry bugs in software systems,” SOSP 2024. https://www.microsoft.com/en-us/research/uploads/prod/2024/08/SOSP_2024__Detecting_Retry_Bugs_in_Software_Systems-1.pdf

[12] “CAFault,” USENIX ATC 2025. https://www.usenix.org/system/files/atc25-chen-yuanliang.pdf

[13] Amazon Web Services, “Amazon S3 multipart upload quotas.” https://docs.aws.amazon.com/AmazonS3/latest/userguide/qfacts.html

[14] Amazon Web Services, “Amazon S3 multipart upload overview.” https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html

[15] Google Cloud, “Cloud Storage resumable uploads.” https://docs.cloud.google.com/storage/docs/resumable-uploads

References were checked on 25 October 2026. Online Python references describe the 3.13 series; the locally tested interpreter is Python 3.13.13.

APPENDIX: INDIVIDUAL TRIAL MEASUREMENTS

All rows use a payload of 524,288 bytes. The time column reports completion time for verified successes and time to failure for Basic trials in the outage scenario. Extra data is expressed in KiB (1,024 bytes). N/A denotes an unavailable recovery value.

TABLE XII
INDIVIDUAL TRIAL MEASUREMENTS, ROUNDED FOR DISPLAY

Scenario	Strategy	Trial	Outcome	Time (s)	Extra (KiB)	Retries	Recovery (s)
Stable	Basic	1	Verified	2.14	0.0	0	N/A
Stable	Retry	1	Verified	2.11	0.0	0	N/A
Stable	Resumable	1	Verified	2.61	0.0	0	N/A
Stable	Basic	2	Verified	2.12	0.0	0	N/A
Stable	Retry	2	Verified	2.12	0.0	0	N/A
Stable	Resumable	2	Verified	2.62	0.0	0	N/A

Scenario	Strategy	Trial	Outcome	Time (s)	Extra (KiB)	Retries	Recovery (s)
Stable	Basic	3	Verified	2.12	0.0	0	N/A
Stable	Retry	3	Verified	2.12	0.0	0	N/A
Stable	Resumable	3	Verified	2.61	0.0	0	N/A
Slow	Basic	1	Verified	16.47	0.0	0	N/A
Slow	Retry	1	Verified	16.46	0.0	0	N/A
Slow	Resumable	1	Verified	16.95	0.0	0	N/A
Slow	Basic	2	Verified	16.46	0.0	0	N/A
Slow	Retry	2	Verified	16.47	0.0	0	N/A
Slow	Resumable	2	Verified	16.95	0.0	0	N/A
Slow	Basic	3	Verified	16.45	0.0	0	N/A
Slow	Retry	3	Verified	16.47	0.0	0	N/A
Slow	Resumable	3	Verified	16.96	0.0	0	N/A
Outage	Basic	1	Failed	2.00	484.0	0	N/A
Outage	Retry	1	Verified	9.95	488.0	5	2.95
Outage	Resumable	1	Verified	8.51	4.0	5	1.02
Outage	Basic	2	Failed	2.01	488.0	0	N/A
Outage	Retry	2	Verified	9.94	484.0	5	2.94
Outage	Resumable	2	Verified	8.49	4.0	5	1.01
Outage	Basic	3	Failed	2.01	484.0	0	N/A
Outage	Retry	3	Verified	9.95	488.0	5	2.95
Outage	Resumable	3	Verified	8.50	4.0	5	1.00

Raw exports retain full precision.